

Advanced Lesson 4 - Exercises: Graphics Mikey Ferguson MATH 7013 - Advanced Mathematical Modeling

```
Clear[a, b, f, g, x, y, u, v, someData];
```

1: ParametricPlot with MeshShading, from the Documentation Center

Parts 1 to 3 are a walk through the Documentation Center page for ParametricPlot. Parts 4 and 5 are below: a MeshShading example copied out of that page, and then the same example with the parameter interval moved from $0 \leq u \leq 4\pi$ to $2\pi \leq u \leq 8\pi$.

MeshShading takes a list of colours (or None) and paints the successive mesh segments of the curve with them in rotation, so the curve is drawn as an alternating dashed ribbon rather than one uniform line.

```
ParametricPlot[{u Cos[u], u Sin[u]}, {u, 0, 4 Pi}, Mesh -> 20, MeshShading -> {Red, Blue}]
```

```
ParametricPlot[{u Cos[u], u Sin[u]}, {u, 2 Pi, 8 Pi}, Mesh -> 20, MeshShading -> {Red, Blue}]
```

Moving the interval does two things at once: the spiral now starts one full turn out from the origin rather than at it, and it runs to four turns rather than two, so the whole picture rescales and the inner hole appears.

2: Four trigonometric functions on one graph

```
Plot[{Sin[x], Cos[x], Tan[x], Cot[x]}, {x, 0, 2 Pi}]
```

Tangent and cotangent have poles inside this interval, so the automatic vertical range is enormous and sine and cosine are squashed flat against the axis. That is the problem Exercise 3 fixes.

3: Restricting the vertical range

```
Plot[{Sin[x], Cos[x], Tan[x], Cot[x]}, {x, 0, 2  $\pi$ }, PlotRange  $\rightarrow$  {-3, 3}]
```

4: Giving each curve its own thickness

```
Plot[{Sin[x], Cos[x], Tan[x], Cot[x]}, {x, 0, 2  $\pi$ }, PlotRange  $\rightarrow$  {-3, 3}, PlotStyle  $\rightarrow$ 
{Thickness[0.01`], Thickness[0.005`], Thickness[0.003`], Thickness[0.001`]}]
```

PlotStyle takes a list matched up with the list of functions, in order: sine gets the thickest line, cotangent the thinnest. Thickness is measured as a fraction of the overall width of the graphic, not in points, so the proportions survive any resizing.

5: Labelling the plot

```
Plot[{Sin[x], Cos[x], Tan[x], Cot[x]}, {x, 0, 2  $\pi$ }, PlotRange  $\rightarrow$  {-3, 3}, PlotStyle  $\rightarrow$ 
{Thickness[0.01`], Thickness[0.005`], Thickness[0.003`], Thickness[0.001`]},
PlotLabel  $\rightarrow$  "sin(x), cos(x), tan(x), cot(x)"]
```

6: The label in 12-point Arial Bold

```
Plot[{Sin[x], Cos[x], Tan[x], Cot[x]}, {x, 0, 2  $\pi$ }, PlotRange  $\rightarrow$  {-3, 3}, PlotStyle  $\rightarrow$ 
{Thickness[0.01`], Thickness[0.005`], Thickness[0.003`], Thickness[0.001`]},
PlotLabel  $\rightarrow$  Style["sin(x), cos(x), tan(x), cot(x)",
FontFamily  $\rightarrow$  "Arial", FontWeight  $\rightarrow$  Bold, FontSize  $\rightarrow$  12]]
```

Style wraps any expression in typesetting directives. The three options used here are exactly the three the target picture calls for. If Arial is not installed the front end substitutes its nearest sans-serif face, which changes the look but not the layout.

7: Adding a frame - the finished graph

```
Plot[{Sin[x], Cos[x], Tan[x], Cot[x]}, {x, 0, 2  $\pi$ }, PlotRange  $\rightarrow$  {-3, 3}, PlotStyle  $\rightarrow$ 
  {Thickness[0.01`], Thickness[0.005`], Thickness[0.003`], Thickness[0.001`]},
  PlotLabel  $\rightarrow$  Style["sin(x), cos(x), tan(x), cot(x)",
    FontFamily  $\rightarrow$  "Arial", FontWeight  $\rightarrow$  Bold, FontSize  $\rightarrow$  12], Frame  $\rightarrow$  True]
```

Frame $\rightarrow$ True draws a box with ticks on all four sides and moves the labels out to the frame, replacing the default crossed axes. This matches the reference picture in the exercise.

8: Plotting data against a fitted curve

```
SeedRandom[20 250 825];
someData = Table[{0.1` i, (0.1` i)2 + 0.5` (RandomReal[] - 0.5`)}, {i, 1, 20}]
```

SeedRandom is not in the professor's version of the exercise. It is here so that the data, the least-squares fit in Exercise 9 and the pictures in Exercises 10 and 11 all refer to the same twenty points on every re-run of this notebook. Delete that line and the numbers change every evaluation.

```
plotData = ListPlot[someData, PlotStyle  $\rightarrow$  PointSize[0.02`]]
Clear[f];
f[x_] = x2;
plotF = Plot[f[x], {x, 0, 2}, PlotStyle  $\rightarrow$  Thickness[0.01`]]
Show[plotData, plotF]
```

PointSize and Thickness are both fractions of the overall graphic width, so 2% and 1% mean what the exercise says regardless of display size. Show overlays finished graphics objects; it does not recompute them, it just composites them onto one set of axes.

9: A least-squares fit using the basis {1, E^x}

```
Clear[g, x];
g = Fit[someData, {1, ex}, x]
```

Fit takes the data, a list of basis functions, and the variable they are written in. It returns the linear combination of those basis functions that minimises the sum of squared residuals - here a constant plus a multiple of E^x. Nothing about Fit requires the basis to be polynomial; it is linear least squares in the coefficients, not in x.

10: Plotting the fit, dashed

```
plotG = Plot[g, {x, 0, 2}, PlotStyle → {Thickness[0.009`], Dashing[{0.02`, 0.02`]}}]
```

Dashing takes a list of segment lengths, again as fractions of the graphic width: 2% drawn, 2% blank, repeating. Thickness and Dashing are combined by putting both directives in one list.

11: All three on one graph

```
Show[plotData, plotF, plotG]
```

The parabola and the exponential fit track each other closely over [0, 2] and neither passes through the scatter exactly - which is the honest picture of a least-squares fit to noisy data.

12: A parametric rose

```
ParametricPlot[{Cos[7 t] Cos[11 t], Cos[7 t] Sin[11 t]},  
  {t, 0, 2 π}, PlotStyle → Thickness[0.01`], AspectRatio → 1]
```

AspectRatio -> 1 forces equal vertical and horizontal scaling, without which the figure would be stretched to the default golden-ratio box and the rotational symmetry would be destroyed. The variable is named t rather than theta only to keep the input in plain ASCII.

13: Sin[x/Sin[y]] - the same surface eight ways

```
Plot3D[Sin[ $\frac{x}{\sin[y]}$ ], {x, -π, π}, {y, -π, π}]
```

Part 1. Trouble is expected wherever $\sin[y] = 0$ - that is at $y = -\pi$, $y = 0$ and $y = \pi$. The quotient $x/\sin[y]$ blows up there, so the argument of the outer sine sweeps through unboundedly many periods in an arbitrarily small neighbourhood of those lines. The function oscillates infinitely fast, and no finite sampling grid can represent it. At exactly $y = 0$ the expression is indeterminate.

```
Plot3D[Sin[ $\frac{x}{\sin[y]}$ ], {x, -π, π}, {y, -π, π}, PlotPoints → 20]
```

```
Plot3D[Sin[ $\frac{x}{\sin[y]}$ ], {x, -π, π}, {y, -π, π}, PlotPoints → 50, Mesh → False]
```

```
Plot3D[Sin[ $\frac{x}{\sin[y]}$ ], {x, - $\pi$ ,  $\pi$ }, {y, - $\pi$ ,  $\pi$ }, PlotPoints → 200, Mesh → False]
```

Parts 2 to 4. Raising PlotPoints resolves more of the structure away from the singular lines and none of it at them - the oscillation near $y = 0$ always outruns the grid. Part 4 is where the cost becomes visible: 200 by 200 is forty thousand sample points and the evaluation takes noticeably longer for a picture that is no more truthful near the singularities.

```
DensityPlot[Sin[ $\frac{x}{\sin[y]}$ ], {x, - $\pi$ ,  $\pi$ }, {y, - $\pi$ ,  $\pi$ }, Mesh → False]
```

```
DensityPlot[Sin[ $\frac{x}{\sin[y]}$ ], {x, - $\pi$ ,  $\pi$ }, {y, - $\pi$ ,  $\pi$ }, PlotPoints → 20, Mesh → False]
```

```
DensityPlot[Sin[ $\frac{x}{\sin[y]}$ ], {x, - $\pi$ ,  $\pi$ }, {y, - $\pi$ ,  $\pi$ }, PlotPoints → 50, Mesh → False]
```

```
DensityPlot[Sin[ $\frac{x}{\sin[y]}$ ], {x, - $\pi$ ,  $\pi$ }, {y, - $\pi$ ,  $\pi$ }, PlotPoints → 200, Mesh → False]
```

Parts 5 to 8. The density plot has the same difficulty for the same reason, and it shows it more honestly: near $y = 0$ and $y = \pm \pi$ the shading degenerates into aliasing artefacts - moire bands that are an artefact of the sampling grid, not features of the function.

Part 9. Describing the behaviour: away from $y = 0$ and $y = \pm \pi$ the surface is a smooth family of ripples, with the ripples in x getting wider as $|\sin[y]|$ grows, because the effective frequency in x is $1/\sin[y]$. As y approaches any zero of $\sin[y]$ that frequency diverges and the function oscillates between -1 and $+1$ infinitely often in any neighbourhood of the line. The function stays bounded in $[-1, 1]$ everywhere - it is a sine, after all - but it has no limit as y approaches 0 , and no continuous extension across those three lines. The correct conclusion from these eight pictures is that no amount of resolution will ever render this function faithfully near $y = 0$, and the way to know that is analysis, not a bigger PlotPoints.

14: Manipulate - dynamic 2-D and 3-D plots

The three cells below are Manipulate expressions. They build correctly under a plain kernel but the sliders are only live inside a Wolfram front end (Mathematica or Wolfram Player). Evaluated through Wolfram Engine under Jupyter they render as a static snapshot at the initial parameter values.

```
Manipulate[Plot[E^(-a x), {x, 0, 1}], {a, 0, 10}]
```

Part 2. The plot jumps as the slider moves because the automatic vertical range is recomputed for each value of a . Fixing PlotRange to All - or to an explicit interval - pins the axes so the curve moves against a fixed frame.

```
Manipulate[Plot[E^(-a x), {x, 0, 1}, PlotRange -> All], {a, 0, 10}]
```

```
Manipulate[
  Plot3D[E^(-a x - b y), {x, 0, 1}, {y, 0, 1}, PlotRange -> All],
  {a, 0, 10}, {b, 0, 10, 2}]
```

Part 3. A control specified as {a, 0, 10} is continuous; adding a step, as in {b, 0, 10, 2}, makes it discrete - the slider snaps to 0, 2, 4, 6, 8, 10 and nothing between.

15: A dynamic algebraic calculator

```
Manipulate[Expand[(x + y)^n], {n, 0, 10, 1}]
```

Same discrete-control mechanism as above: the step of 1 restricts n to integers, which is the only thing Expand can usefully do with an exponent here.

16: Analysing $2x^2 + 3x^3 y == 4x^4 + 5y + 6$

```
Clear[f, x, y];
sol16 = Solve[2 x^2 + 3 x^3 y == 4 x^4 + 5 y + 6, y]
```

```
f[x_] = y /. First[sol16]
```

Parts 1 and 2. The equation is linear in y, so there is a single solution, and extracting the right-hand side of that one rule gives an explicit rational function of x. Immediate assignment is right here: the expression is fixed and needs no re-evaluation per call.

```
Plot[f[x], {x, -4, 4}]
```

Part 3. The plot has a vertical asymptote where the denominator $3x^3 - 5$ vanishes, near $x = 1.19$.

```
Simplify[f'[x]]
```

```
Plot[f'[x], {x, -4, 4}]
```

Parts 4 and 5. The derivative is another rational function, with a double pole at the same place.

```
Solve[f'[x] == 0, x]
```

Part 6. Setting the derivative to zero clears the denominator and leaves a sixth-degree polynomial. Degree six has no solution by radicals in general, so Solve returns Root objects: exact algebraic numbers specified by the polynomial they satisfy and an index saying which root is meant. They are exact and perfectly usable, but nobody can read them.

```
NSolve[f'[x] == 0, x]
```

Part 7. NSolve returns the same six roots numerically: two complex conjugate pairs' worth of complex values and four real ones.

```
roots16 = Chop[x /. NSolve[f'[x] == 0, x]]
```

```
realRoots16 = Select[roots16, Im[#1] == 0 &]
```

```
TableForm[Table[{r, Chop[f'[r]], Chop[f''[r]],  
  If[f''[r] < 0, "local maximum", "local minimum"]}, {r, realRoots16}],  
  TableHeadings -> {None, {"x", "f'[x]", "f''[x]", "classification"}}]
```

Part 8. The six roots split into two complex ones and four real ones, and the four real ones do agree with the picture in Part 5 - each is a place where the derivative curve crosses zero. Classifying them by the sign of the second derivative gives, to four figures: $x = -0.8427$ and $x = 0.2671$ are local maxima of f , while $x = 0$ and $x = 1.9615$ are local minima. So the two real roots corresponding to local maxima are -0.8427 and 0.2671 .

One caution when reading the derivative plot. The sign of f' also flips across the vertical asymptote near $x = 1.19$, and that flip is not a critical point - the derivative is undefined there, not zero, which is exactly why NSolve does not return it.